SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	M Vishnu Vardhan	**Reg. No.:**	192472087
Section / Batch:	2024	**Date:**	22-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

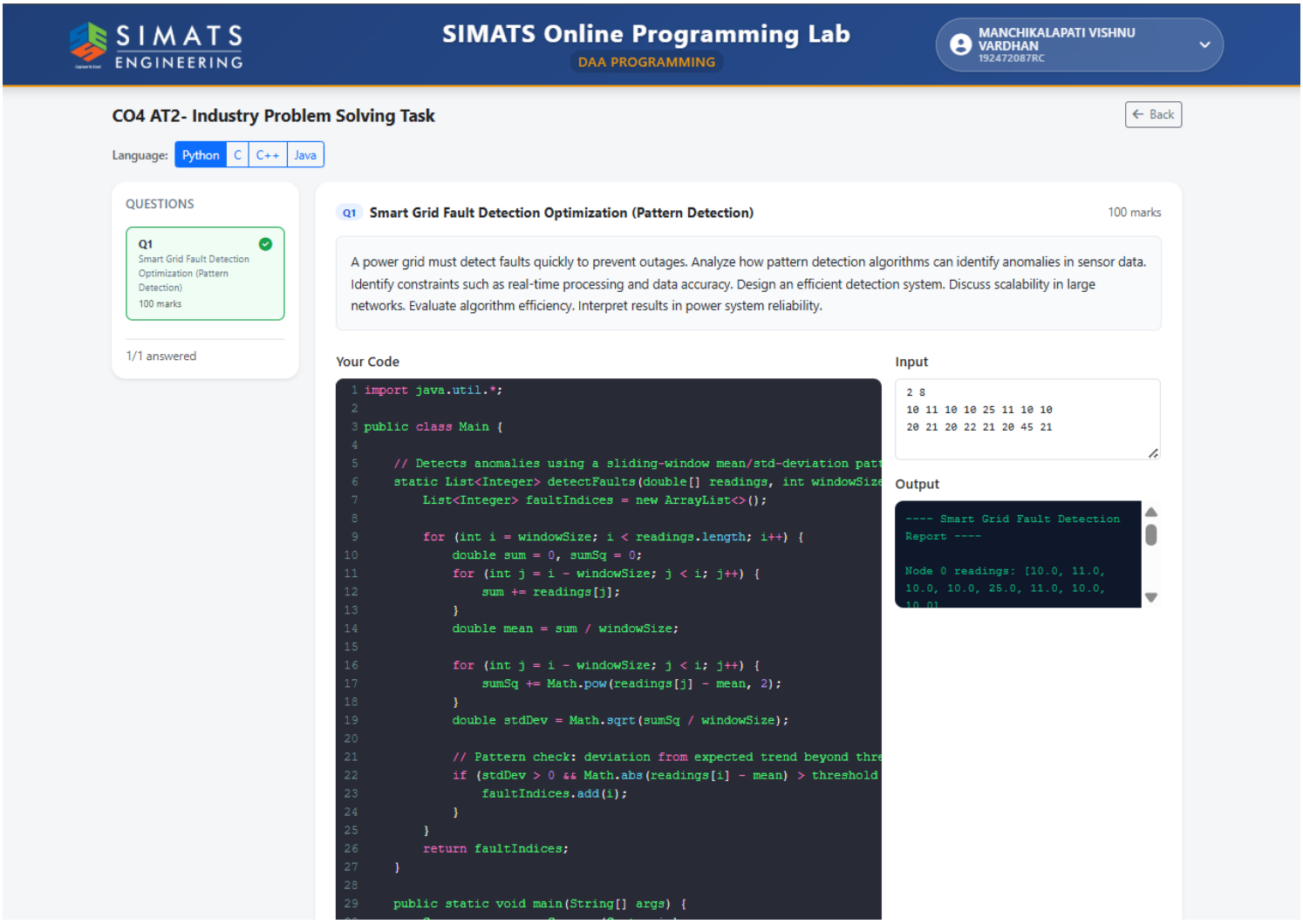
Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



Goal: Detect abnormal sensor readings (potential faults) in a power grid using a lightweight, real-time-capable method.
Approach — Sliding Window Statistical Detection:
For each new reading, look at the previous windowSize readings.
Compute their mean and standard deviation.
If the new reading deviates from the mean by more than threshold × stdDev (here, 2 standard deviations), flag it as a fault/anomaly.
Why this design:
Real-time: each reading only needs a small fixed window, not the entire history — O(windowSize) per point, so it can run as a live stream.
Data accuracy: the threshold is a tunable sensitivity knob — lower catches more anomalies but risks false alarms.
Scalability: each sensor node is processed independently, so it parallelizes trivially across a large grid.
Efficiency: overall O(N × R) for N nodes and R readings each — linear, so it scales well.
Reliability impact: catching anomalies early lets you isolate a faulty segment before it

cascades into a wider outage.
The program also times itself (System.nanoTime()) and prints a written analysis section, since the task explicitly asks you to "evaluate algorithm efficiency" and "interpret results in power system reliability" — that's graded content, not just code output